

- A. What is the difference between EREW, CREW, and CRCW models? Provide an example for each of them. (1)

EREW (Exclusive Read Exclusive Write):

No two processors can read from or write to the same memory cell at the same time.

Example: Copying an array

CREW (Concurrent Read Exclusive Write)

Multiple processors can read the same memory cell simultaneously, but only one can write to a given cell at a time.

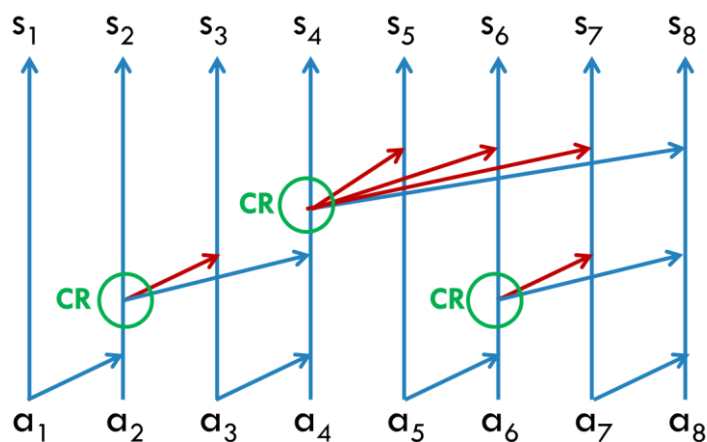
Example: Matrix-vector multiplication

CRCW (Concurrent Read Concurrent Write)

Multiple processors can read from and write to the same memory cell simultaneously. Write conflicts are resolved by a rule (e.g., common, priority, arbitrary).

Example: DNF computation where the variables read can be any of the input variables.

- B. How can the prefix sum of an array be computed on a CREW PRAM using idle processors from the sum algorithm? (1)



- C. Given an algorithm with $T_1(n)=n^2$ and $T_p(n)=n^2/p$, analyze the speedup and efficiency. What can you conclude? (1)

Speedup: $S_{up} = T_1(n) / T_p(n) = p$

Efficiency: $E_p = T_1(n) / (p T_p(n)) = 1$

Conclusion: The algorithm achieves **linear speedup** in the best case and **perfect efficiency**.

- D. What is the main conceptual difference between Amdahl's Law and Gustafson's Law? (1)

Amdahl assumes a fixed problem size and shows limits to speedup.

Gustafson assumes a fixed execution time and scales the problem size, demonstrating that the speedup can grow linearly with the number of processors.

- A. Explain the difference between superscalar execution and SIMD execution. (1)

Superscalar execution allows a processor to execute multiple independent instructions from the same instruction stream in parallel. SIMD (Single Instruction, Multiple Data) executes the same instruction on multiple data elements simultaneously. Instruction stream coherence is crucial for SIMD because all ALUs execute the same instruction; divergence leads to inefficiency. Superscalar processors, however, can handle different instructions independently.

- B. Why does multi-threading help hide memory latency, and how many threads are needed to achieve full utilization if each thread performs 3 arithmetic operations with a latency of one cycle, followed by a memory load with 12-cycle latency? Please motivate the answer. (1)

Multi-threading hides memory latency by allowing the processor to switch to another thread when one is stalled. To hide a 12-cycle latency with 3 arithmetic instructions per thread, 5 threads are needed (3 instructions $\times$ 5 threads = 15 cycles, covering the 12-cycle stall).

- C. What does it mean for a computation to be bandwidth-bound, and why is vector multiplication of large arrays an example of this? (1)

A computation is bandwidth-bound when its performance is limited by the rate at which data can be transferred from memory, not by the processor's compute capability. Vector multiplication involves multiple memory accesses per arithmetic operation, saturating memory bandwidth before compute units are fully utilized.

- D. Write a short pseudocode that would result in worst-case SIMD efficiency on an 8-wide SIMD processor due to divergent execution. (1)

```
forall (int i from 0 to N) {  
    float t = x[i];  
    if (i % 8 == 0) {  
        t = t * 2.0;  
    } else if (i % 8 == 1) {  
        t = t + 3.0;  
    } else if (i % 8 == 2) {  
        t = t / 4.0;  
    } else if (i % 8 == 3) {  
        t = t - 5.0;  
    } else if (i % 8 == 4) {  
        t = sqrt(t);  
    } else if (i % 8 == 5) {  
        t = log(t);  
    } else if (i % 8 == 6) {  
        t = exp(t);  
    } else {  
        t = sin(t);  
    }  
    y[i] = t;  
}
```

- A. Please describe shared address space model in terms of communication abstraction and required hardware support. (2)

See slides lecture-4_progmodels pp. 30-37.

- B. Why must CUDA allocate execution contexts for all threads in a block before execution begins? (2)

Because CUDA threads in a block may synchronize (e.g., using `__syncthreads()`), all threads must be live concurrently to avoid deadlocks and ensure correct execution. This requires allocating resources (registers, shared memory) for all threads upfront.

See slide lecture-5_gpuarch page 67.

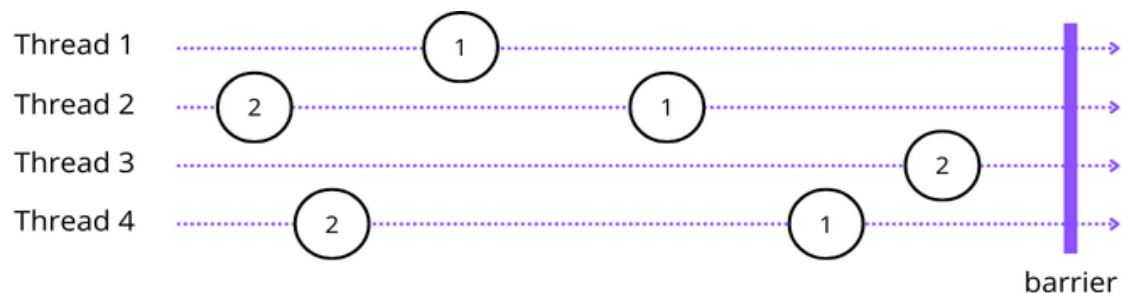
- A. What is the difference between joining and waiting on a barrier in Pthreads? (1)

Joining synchronizes the execution of two threads, the joined thread is destroyed. A barrier synchronizes the execution of a group of threads, all of them continue working after the last one reaches the barrier.

- B. Give two examples of OpenMP environment variables and how they influence program execution. (1) **This question can be skipped if you got 1 or 3 points in the first challenge.**

Examples: `OMP_NUM_THREADS` to set the number of threads if not specified in the program, `OMP_NESTED` to enable/disable nested parallelism, `OMP_CANCELLATION` to enable/disable cancellation... Environment variables influence the execution of any OpenMP program launched in that environment.

- C. Describe how each thread in the following diagram behaves at runtime as it encounters points marked with 1, representing `pragma omp cancel`, and with 2, representing `pragma omp cancellation point`. (2) **This question can be skipped if you got 2 or 3 points in the second challenge.**



In order:

Thread 2 reaches a cancellation point but no thread has requested cancellation, so it continues.

Thread 4 reaches a cancellation point but no thread has requested cancellation, so it continues.

Thread 1 reaches a cancel directive, sets the cancellation flag, stops executing, waits at the barrier.

Thread 2 reaches a cancel directive, sees that the cancellation flag has been already set, stops executing, waits at the barrier.

Thread 4 reaches a cancel directive, sees that the cancellation flag has been already set, stops executing, waits at the barrier.

Thread 3 reaches a cancellation point, sees that the cancellation has been set, stops executing, waits at the barrier.